

🌐 Railway vs Render vs Laravel Cloud vs Vercel/Cloudflare — مقارنة استضافة مشروع وداد

🚀 المتطلبات الفعلية للمشروع (نقطة الانطلاق لكل مقارنة)

كامل محتاج Laravel قبل أي مقارنة، مشروع وداد مش "موقع استاتيك بسيط" — هو تطبيق

المتطلب	ليه مهم
API 24/7 يشغل	تسجيل دخول، إلخ، Paymob من Webhook، يستقبل حجوزات
Queue Worker دائم الشغل	Jobs كل شوية — الـ cron مش (<code>ProcessChatbotMessageJob</code>), (<code>ProcessLabTestJob</code>), (<code>UploadChatbotDocumentJob</code>)... لازم تتعالج فوراً أو شبه فوراً
Redis	Cache + Queue + Session driver
MySQL	Postgres ولا SQLite مش، 81 migration
Laravel Scheduler	(<code>schedule:run</code>) كل دقيقة (تذكير مواعيد، Sync Google Fit، Wakeup APIs)
تخزين ملفات	Chatbot RAG مستندات، (OCR) رفع تحاليل طبية
معاملات مالية حقيقية	Hobby project يعني المشروع تجاري رسمياً، مش — Paymob Live

هذه النقطة الأخيرة (تجاري) هتأثر بشكل مباشر على اختيار منصة الفرونت، وهنشرها بالتفصيل تحت

1 Backend + Frontend + DB) الخيار الأول: منصة واحدة تستضيف كل حاجة

على منصة واحدة زي (وحتى ملفات الفرونت المبنية، MySQL، Redis، Queue Worker، API) services يعني كل الـ Railway أو Laravel Cloud.

✅ المميزات بالتفصيل

1. واحد لكل حاجة، لوج واحد تراجع، فاتورة واحدة تتابعها. ده فرق كبير لو الفريق صغير (أو dashboard — إدارة موحدة مطور واحد) زي حالتك.
2. (Private) بيحصل جوه شبكة خاصة Redis والـ MySQL والـ API شبكة داخلية أسرع وأمن — الاتصال بين للمنصة نفسها، مش عبر الإنترنت العام. ده معناه (Network)
 - (اللي أصلاً شوية بطيئة OCR والـ Chatbot مهم لأداء الـ) أقل Latency
 - مش معرّضة لأي حد بره المنصة DB أمان أعلى — الـ

3. تلقائي لكل الخدمات مع deploy يعمل (git push) مباشرة، أي GitHub مبسّط — أغلب المنصات دي بتربط بـ CI/CD. بعض بدون سكريبتات إضافية.
4. معقدة، مفيش قلق من اختلاف التوقيت بين نشر تحديث في الفرونت CORS مفيش تعقيد "دومين بيكلم دومين" — مفيش مشاكل. وتحديث في الباك إند.
5. بشكل منفصل، بتحسب فاتورة واحدة Railway + Vercel تسعير مجمّع أسهل تتوقعه — بدل ما تحسب فاتورة.

⚠ العيوب بالتفصيل

1. عالمي CDN مش متخصصة في توزيع ملفات استاتيكية عبر Railway أداء الفرونت الاستاتيكي أقل تحسينًا — منصات زي يعني تحميل صفحة الفرونت للمستخدم في مصر ممكن يبقى أبطأ شوية. Cloudflare أو Vercel بنفس كفاءة.
2. الموقع كله (فرونت، Downtime أو Incident لو المنصة عندها) — نقطة فشل واحدة (Single Point of Failure) (+ باك) بيقع مع بعض. لو كانوا منفصلين، على الأقل الفرونت (المبني مسبقًا) يفضل شغال.
3. من الصفر Build pipeline صعوبة توسّع لاحقًا — لو حبيت تنقل الفرونت لمنصة ثانية بعدين، هتحتاج تعيد ضبط الـ.
4. غير ثابتة — الفاتورة بتتغير حسب حركة الموقع، مش سعر ثابت متوقع كل شهر usage-based تكلفة.

👤 أنسب لمين؟

مشروع في مرحلة الإطلاق الأولى (زي وداد الآن)، فريق صغير أو مطور واحد، الأولوية للبساطة والسرعة في الإطلاق أكثر من تحسين كل جزء لحاله.

2 منفصل Frontend (Vercel/Cloudflare) + Backend (Railway/Render/Laravel Cloud) — الخيار الثاني: تقسيم

✅ المميزات بالتفصيل

1. Edge Servers بتتوزع من (npm run build) بعد (HTML/CSS/JS) أداء أفضل للفرونت — الملفات الاستاتيكية قريبة جغرافيًا من المستخدم، فالتحميل الأول للموقع أسرع.
2. إنه بيستخدم (config/cors.php) الكود عندك أصلًا مجهّز للفكرة دي — لاحظت في.

php

```
'allowed_origins' => [  
    env('FRONTEND_URL', 'http://localhost:5173'),  
    env('APP_URL', 'http://localhost:8000'),  
],
```

مستقل Deploy. صح. 3 .env ده معناه إن فصل الدومينين مبني في الكود من الأساس، مش هتحتاج تعديل جوهري — بس تظبط الـ Deploy ما يستدعيش إعادة نشر الباك إند، والعكس. ده بيقال مخاطر كل (UI bug تغيير تصميم، إصلاح) لكل جزء — تحديث الفرونت مفيد جدًا لاختبار Pull Request بتديك رابط تجريبي منفصل مع كل Vercel منصات زي — 4. Preview Deployments مرونه اختيار الأنسب لكل جزء — 5. مرونه اختيار الأنسب لكل جزء — تقدّر تختار أخص وأنسب منصة للفرونت، وأنسب منصة لتعديل الفرونت قبل ما تنزل للإنتاج. 5. مرونه اختيار الأنسب لكل جزء — تقدّر تختار أخص وأنسب منصة للفرونت، وأنسب منصة للبلاك إند، من غير ما تكون مقيد بمنصة واحدة تعمل الاثنين بجودة متوسطة.

⚠ العيوب بالتفصيل

1. بالظبط Vercel/Cloudflare الباك إند يطابق دومين env في `FRONTEND_URL` دقيق مطلوب — لازم CORS ضبط
Console. في الـ CORS وإلا هتظهر أخطاء، (وبدون / في الآخر، https بروتوكول)
2. إضافي بسيط — أي طلب من الفرونت للباك إند بيعدي بين دومينين مختلفين (مش نفس السيرفر)، فيه فرق بسيط جدًا Latency
في الزمن (عادة غير محسوس للمستخدم لكنه موجود)
3. إدارة حسابين/فواتير منفصلة — تتابع استخدام واستقرار منصتين بدل واحدة.
4. لو حصلت مشكلة، محتاج تحدد هل هي في الفرونت، في الباك إند، ولا في الاتصال بينهم — Debugging تعقيد إضافي وقت الـ
(CORS/Network).

🔴 وشروط الاستخدام التجاري Vercel: تنبيه مهم جدًا لمشروعك تحديدًا

تنص على إنها للاستخدام الشخصي وغير (Terms of Service) شروط استخدامها الرسمية (Hobby) المجانية Vercel خطة
بسعر Pro من الناحية الرسمية لازم يكون على خطة (شغال بالفعل Paymob وعندك تكامل) التجاري فقط. أي موقع بياخد فلوس فعليًا
20\$/شهر — وده لوحده بيتخطى ميزانيتك بالكامل

- لكن ده يفضل مخالفة رسمية، Vercel ومفيش إنفاذ فعلي صارم من Hobby عمليًا؛ ناس كثير بتشغل مشاريع تجارية صغيرة على
للشروط ممكن (نظريًا) يترتب عليها إيقاف الحساب بدون إنذار
- **Cloudflare Pages: البديل الأنسب لك**
 - bandwidth مجاني بالكامل، بدون حد أقصى لل
 - شروط الاستخدام بتسمح صراحة بالمشاريع التجارية
 - Vercel لأنه أصلاً مش محتاج ميزات، (SPA build) عادي React/Vite لموقع Vercel أداء مطابق تقريبًا لـ
ملفات (Vite build → عادي React مشروعك فرونت — (Next.js SSR, Edge Functions) المتقدمة
(استاتيكية)
 - للفرونت — نفس الفكرة، بدون مخاطرة الشروط التجارية، Vercel بدل Cloudflare Pages التوصية: استخدم
وبتوفر الميزة انية كلها للباك إند

3 [الباك إند] Railway vs Render vs Laravel Cloud: جدول المقارنة التفصيلي

المعيار	Railway	Render	Laravel Cloud
MySQL دعم	مُدارة MySQL خدمة ☒	غير موجود أصلاً كخدمة مُدارة ☒	مُدار بالكامل، — "MySQL Flex" ☒
مُدَار	one-click، بالكامل	لازم Postgres. — المتاح فقط يدويًا جوه MySQL تشغل وتديره Docker container بنفسك (نسخ احتياطي، تحديثات، إلخ)	نفس المحرك اللي مشروعك يستخدمه فعليًا
Queue	منفصل Service ☒	نوع خدمة أساسي ☒	نظام — "Managed Queues" ☒
Worker دائم	يشتغل <code>queue:work</code> ☒ باستمرار طول ما الخدمة شغالة	(Background Worker)،	بتشتغل تلقائيًا Workers مختلف جوهريًا: الـ فعالية بس، وترجع صفر Jobs وقت وجود

المعيار	Railway	Render	Laravel Cloud
		لكن غير متاح على الخطة المجانية (Starter) — يبدأ من 7\$/شهر	لما الطابور يفضى — (Scale-to-Zero) يعني تدفع على "المعالجة"، مش على "وقت الانتظار"
Redis	✓ Add-on مُدار، إعداد بسيط من الداشبورد	✓ "Key Value" متوافق مع Redis لكن أرخص خطة مدفوعة، تبدأ من ~10\$/شهر لوحدها	✓ "Valkey" (Redis API متوافق مع) متضمن ضمن نفس الخطة، بدون تكلفة — منفصلة إضافية عند الاستخدام البسيط
Laravel Scheduler	✓ Cron support على service مستوى كل	✓ Cron Jobs خدمة منفصلة	✓ php artisan schedule:run مباشرة بدون إعداد يدوي
مصمم خصيصًا لـ Laravel	✗ تدعم أي لغة) منصة عامة (Nixpacks عبر	✗ منصة عامة	✓ نفسه — Laravel مبني ومُدار من فريق Horizon أقل إعدادات يدوية، تكامل مباشر مع Scheduler والـ Jobs والـ
تخزين ملفات (OCR / مستندات Chatbot)	منفصل أو Volume يحتاج خارجي S3-compatible	منفصل Persistent Disk يحتاج خارجي S3-compatible أو	✓ S3-compatible object storage (Cloudflare R2) متكامل من نفس اللوحة
دومين مخصص	✓ مجاني	✓ مجاني	✓ 10 — (Starter) متضمن من أول خطة دومينات
سهولة تطبيق الـ Checklist (بتاعك 22 بند)	متوسطة — بعض الإعدادات يدوية (Supervisor-equivalent)	مشكلة) متوسطة إلى صعبة (بتضيف خطوة كاملة MySQL)	سهلة — أغلب البنود (Queue, Scheduler, Redis, SSL, Domain) جاهزة من الداشبورد مباشرة
أقرب تقدير تكلفة شهرية واقعية لمشروع وداد	شهر تقريبًا/8-15\$ أساسي + \$5 (Hobby API + Worker + Redis + MySQL (الفعلي	Web \$7 + Worker \$7 + Redis \$10 + MySQL يدوي 7\$ تقريبًا → بره ميزانيتك بوضوح	شهر تقريبًا حسب حركة الموقع الفعلية/5-15\$ — Starter 5\$ credit بشمل 5\$ مجاني الاستخدام

📌 الخلاصة من الجدول

العالية نسبيًا، بتخليه بره نطاق Worker وRedis والمُدار رسميًا، وتكلفة MySQL مش خيار عملي لمشروعك — غياب Render وLaravel Cloud وRailway الميزانية (\$5-10) من الأساس. المقارنة الحقيقية بقت بين

4 مين يكسب؟ — Railway vs Laravel Cloud

Railway	Laravel Cloud
الميزة الأقوى	بناعتك هتشتغل كـ Jobs الـ — Laravel تكامل مباشر ومتخصص مع كل جزء في <code>systemd</code> <code>Supervisor</code> بدون كتابة أي <code>Managed Queue</code> زي المخطط في الملف الأصلي <code>config</code>
التوفير الفعلي في الفلوس	بترجع لصفر تلقائيًا وقت عدم وجود شغل — أوفر فعليًا <code>Managed Queues</code> لمشروع في بداية إطلاقه بحركة متوسطة/متذبذبة في <code>Jobs</code> باستمرار حتى لو مفيش الطابور
مخاطرة تغيير سياسة التسعير	منصة أحدث (أطلقت فبراير 2025) لسه بتتطور وبتضيف مميزات، لكن مدعومة Laravel مباشرة من فريق
سرعة تطبيق الـ 22 بند Checklist	أسرع — أغلب البنود جاهزة من الداشبوردي
	تحتاج بعض الإعداد اليدوي للـ Worker

Laravel → بالكامل **Laravel** بتاعك، وأعلى توافق تقني مع كون المشروع **Checklist التوصية**: لو عايز أقل احتكاك ممكن مع الـ **Cloud** خيار قوي كمان **Railway** → هو الأنسب. لو عايز بديل بسيط وسريع بنفس نطاق الميزانية تقريبًا

5 (شهر/\$5) "Laravel Cloud Starter" تفصيل كامل لخطة

بما إنها المرشح الأقوى للباك إند، خايني أشرحلك كل بند فيها بالتفصيل وأربطه بمشروع ووداد تحديدًا

👛 First month free

للمدة شهر كامل بدون أي تكلفة، وتتأكد إن كل حاجة شغالة صح قبل ما تدفع أي فلوس (OCR، Chatbot، Sandbox تسجيل، حجز، دفع) **Flows** أول شهر مجاني بالكامل — يعني تقدر تنزل المشروع فعليًا، تختبر كل الـ

📺 \$5 per month plus usage

المجانبة (اللي هي أصلًا \$5 شهريًا — يعني عمليًا لو **credits** السعر الأساسي \$5/شهر، وفوقه بتدفع على أي استهلاك زايد عن الـ). استهلاكك في حدود معقولة، ممكن الفاتورة الفعلية تفضل قريبة من \$5-8 بس

📺 Includes \$5 in monthly usage credits

تتجدد تلقائيًا (مش نتراكم لو مستخدمتهاش)، بتغطي **credit** كل شهر بتاخد \$5

- وقت اشتغالها **Managed Queue** الـ + **Compute (API)** الـ
- **MySQL Database** الـ
- **Redis/Valkey cache** الـ

- Storage (R2 لو مستخدم)
- Bandwidth

بالنسبة لمشروع في بداية إطلاقه بعدد مستخدمين محدود، الاستهلاك الفعلي ممكن يقرب جدًا من الـ \$5 دول، يعني الفاتورة الشهرية الحقيقية تكون قريبة من الحد الأدنى (\$5) في الشهور الأولى.

Import your repo, deploy in seconds

Build تلقائيًا وتضبط الـ Laravel والمنصة بتكتشف إنه مشروع repo (Back-end)، بتاعك، تختار الـ GitHub تربط حساب أو سكربت يدوي Dockerfile بدون (إلخ، env مفاتيح، composer install) خطوة بخطوة.

Unlimited applications, environments, builds, & users

بدون حد أقصى، وتقدر تضيف أعضاء فريق (لو حد هينضم للمشروع (مثلاً Staging + Production) تقدر تعمل بيانات متعددة (بعدين) بدون تكلفة إضافية على عدد المستخدمين.

Scale-to-Zero Flex compute

وبيصح في أقل من 500 ملي Traffic، بينام تلقائيًا وقت عدم وجود (App + Compute) دي أهم ميزة تقنية بالنسبة لك: السيرفر: ثانية أول ما يجيله طلب. الفائدة:

- توفير فلوس حقيقي في ساعات قلة الاستخدام (الليل مثلاً)
- Managed بينصح باستخدام Laravel، وقت ما السيرفر بينام (OCR زي معالجة) طويل شغال Job لكن انتبه: لو فيه المستمر ميتقطعش — وده بالضبط الإعداد الافتراضي في خطة Job عشان الـ (عادي Worker مش) بالتحديد Queues Starter.

Spending limits and alerts

تقدر تحدد سقف إنفاق شهري (مثلاً \$10) والمنصة تنبهك أو توقف الخدمة تلقائيًا لو قربت من الحد ده — بيحميك من أي فاتورة مفاجئة لو غير متوقع (مفيد جدًا وانت لسه بتقيس الاستهلاك الحقيقي للمشروع) Traffic حصل.

Managed queues

اللي كانت أكبر عقبة في خطة النشر الأصلية. بدل ما تكتب Queue Worker أهم بند ليك تحديدًا — دي الحل المباشر لمشكلة الـ بتاعتك Jobs الـ، (GCP/AWS زي المذكور في الملف الأصلي لـ) يدوي (Supervisor config) أو (systemd service) (ProcessChatbotMessageJob), (ProcessLabTestJob), (UploadChatbotDocumentJob), (SendAppointmentReminders), (SyncGoogleFitData) بتتشغل تلقائيًا من غير أي إعداد سيرفر، وبتتوسع تلقائيًا (Autoscale) Jobs. لو زاد عدد الـ.

DDoS mitigation

وطلبات (Webhooks (Paymob) عام زي بتاعك بيستقبل API حماية تلقائية من هجمات إغراق السيرفر بالطلبات — مهم لأي تسجيل دخول من الإنترنت المفتوح.

10 custom domains

تقدر تربط حتى 10 دومينات فرعية مختلفة — يكفي بالراحة لسيناريو زي

- (الباك إند) `api.widad.com`
- (لو لوحة تحكم منفصلة) `admin.widad.com`
- Staging دومينات تجريبية لبيئة

📝 1 day log retention

اللوغات (سجلات الأخطاء والطلبات) بتتحتفظ لمدة يوم واحد بس. ده كافٍ للتصحيح السريع وقت حدوث مشكلة، لكن لو عايز أرشيف أطول وأصلاً مذكورة في خطة النشر) **Sentry** محتاج تربط خدمة خارجية زي ،(مهم لتتبع مشاكل نادرة أو مراجعات لاحقة) logs للـ. عشان تحتفظ بالأخطاء لفترة أطول بشكل منفصل عن لوغات المنصة (الأصلية بتاعتك في المرحلة الثالثة).

🔗 Standard support

كافي جداً لمرحلة الإطلاق، ولو المشروع كبر بعدين تقدر — (SLA مش استجابة فورية مضمونة بـ Support تذاكر) دعم فني عادي Pull لكل (Preview Environments) للحصول على دعم أسرع ومميزات إضافية (شهر/\$20) Growth تترقى لخطة Request Worker Clusters، مخصصة Autoscaling 10 replicas حتى.

6 التوصية النهائية

🏆 الأنسب لمشروع وداد بناءً على ميزانيتك ومتطلباتك التقنية:

للفرونت (مجاني) Cloudflare Pages + للباك إند (شهر/\$5 — Starter) Laravel Cloud

السبب:

1. بشكل مُدار بالكامل، بتكلفة داخل ميزانيتك المحددة (\$5-10) Redis والـ Queue Worker يحل مشكلة الـ
2. Render غير مدعوم "زي MySQL" مفهوش مشكلة
3. بتاعتك هيبقى أسرع وأقل عرضة للأخطاء اليدوية Checklist تطبيق الـ 22 بند — Laravel متخصص في
4. (Vercel Hobby بعكس مخاطرة) للفرونت: مجاني بالكامل + يسمح بالاستخدام التجاري رسمياً Cloudflare Pages

في نفس **Railway** → (بضغطة واحدة، مرونة أكبر لأي لغة برمجة مستقبلاً Deploy) البديل القوي: لو حبيت تجربة استخدام أبسط نطاق الميزانية تقريباً.

📝 الخطوة التالية

مطبقة فعلياً على أي من الخيارين دول (بالـ 4 مراحل DEPLOYMENT_PLAN.md) عشان أكتبك الخطة التنفيذية الكاملة: محتاج ترفعلي ملفات المشروع الفعلية المذكورة في طلبك الأصلي، (Railway أو Laravel Cloud)

```
Back-end/.env.example
Back-end/config/{cors,queue,cache,session,filesystems,chatbot,services}.php
Back-end/app/Console/Kernel.php
Back-end/app/Jobs/
Back-end/app/Services/
Back-end/routes/{patient,doctor,admin}.php
Back-end/database/migrations/
Front-End/.env.example
Front-End/vite.config.ts
Front-End/package.json
Back-end/composer.json
```

بدون الملفات دي، أي خطوة تنفيذية هتبقى افتراضية مش مبنية على الكود الحقيقي بتاعك.

تم إعداد هذه المقارنة بناءً على أسعار ومواصفات المنصات المُحقق منها في يوليو 2026 — الأسعار قابلة للتغيير، يُنصح بمراجعة صفحات التسعير الرسمية قبل الالتزام النهائي.